Smart Commerce - Key Features Defense Notes

This document explains the main intelligent features of my Smart Commerce project in a way I can present and defend in front of my supervisor.

The features are explained in three parts: first, a short speech; second, how the feature works; and third, how I built it in this project.

1. AI Demand Forecasting

Speech:

One of the most important features of my project is demand forecasting. In Smart Commerce, demand forecasting helps the admin understand which products may sell more in the near future. The system does not use deep learning. It uses an explainable Python-based forecasting script. The backend reads previous order data, groups product sales into recent weekly history, and sends that data to the Python script. The script calculates predicted units for the next 7, 14, and 21 days. It also shows whether demand is rising, stable, or cooling, and gives a confidence score. This helps the admin make better restocking decisions.

How It Works:

The backend collects all active products from the Product collection.

It collects orders where the status is not cancelled.

For each product, the backend prepares weekly sales history using order item quantities.

The sales history is sent to the Python script called demand_forecast.py.

The Python script calculates a weighted average, so recent sales have more importance than older sales.

It compares earlier sales and recent sales to label the trend as rising, stable, or cooling.

It extracts simple keywords from product names and applies small keyword-based boosts for some product terms.

It returns predicted units, recent sold units, total sold units, trend, keywords, confidence score, and forecast values.

The admin dashboard shows the result as a chart and ranking table.

How I Built It:

In the backend, the main logic is in backend/src/modules/analytics/analytics.service.ts.

The getDemandForecast function collects products and orders, prepares product sales history, and calls the Python forecasting function.

The Python script is backend/scripts/demand_forecast.py.

The backend runs the Python script using Node.js child_process spawn.

If python is not available, the backend tries python3.

If both fail, the backend uses a fallback forecasting function.

The route is protected under /api/analytics/demand-forecast.

Only admin and super_admin roles can access this forecast route.

In the frontend, frontend/app/(admin)/admin/demand-forecast/page.tsx displays the demand forecast page.

The chart is built with Recharts through the DemandForecastChart component.

The forecast report export feature is handled by ForecastReportExport.

Important Defense Point:

If I am asked whether this is deep learning, my answer is no. It is an explainable baseline forecasting method using weighted average, trend detection, keyword extraction, and confidence scoring.

2. Inventory Intelligence

Speech:

Inventory intelligence is another key feature of my system. The purpose is to help admins and vendors understand the stock condition of products. Each product has stock and reorder point values. The system shows current stock, stock status, low-stock condition, and out-of-stock condition. It also allows stock adjustment, and every adjustment is saved as an inventory log. This makes inventory management more controlled and traceable.

How It Works:

Each product stores stock and reorderPoint in the Product model.

The inventory overview API reads products that are not archived.

Products are sorted by stock level so low stock items become easier to notice.

The frontend compares stock with reorderPoint to show statuses like in stock, low stock, or out of stock.

Admins and vendors can adjust stock by entering a stock change value.

The backend calculates stockBefore and stockAfter.

The backend prevents stock from becoming negative by using Math.max(0, stockBefore + delta).

Each stock change is stored in the InventoryLog collection.

The log stores product name, change type, quantity, previous stock, new stock, reason, and who made the change.

How I Built It:

The Product model is in backend/src/models/product.model.ts and includes stock and reorderPoint.

The InventoryLog model is in backend/src/models/inventory.model.ts.

The inventory overview and stock adjustment logic are in backend/src/modules/products/products.service.ts.

The stock adjustment controller is in products.controller.ts.

The protected backend endpoints include /api/products/inventory/overview, /api/products/inventory/logs/:id, and /api/products/:id/stock.

These routes are protected for super_admin, admin, and vendor roles.

The admin inventory page is frontend/app/(admin)/admin/inventory/page.tsx.

The vendor inventory page is frontend/app/(vendor)/vendor/inventory/page.tsx.

The stock adjustment UI is built in frontend/components/vendor/inventory-adjust.tsx.

Important Defense Point:

Inventory intelligence in this project means stock visibility, reorder comparison, low-stock detection, stock adjustment, and stock movement logging. It is not automatic supplier purchasing.

3. Low-Stock Alerts

Speech:

Low-stock alert is a focused feature inside the inventory module. The system identifies products where stock is below a fixed alert threshold. In my backend, the low-stock threshold used for dashboard alerts is 7 units. The admin can see product name, vendor name, SKU, current stock, alert threshold, suggested reorder quantity, and status. This helps the admin quickly find products that need restocking attention.

How It Works:

The analytics overview service searches for active products where stock is less than 7.

It also populates vendor information so the admin can see which vendor provides the product.

If stock is zero, the urgency is critical.

If stock is above zero but below the threshold, the urgency is high.

The recommended order quantity is calculated as threshold minus current stock.

The frontend shows these products in the admin dashboard and in the low-stock product page.

How I Built It:

The low-stock threshold is defined in backend/src/modules/analytics/analytics.service.ts as lowStockThreshold = 7.

The getDashboardOverview function queries active products with stock below this threshold.

It returns inventoryAlerts to the frontend.

The admin dashboard shows these alerts through the InventoryAlerts component.

The dedicated low-stock page is frontend/app/(admin)/admin/low-stock-product/page.tsx.

This page displays a table with product, vendor, SKU, stock, alert below, suggested quantity, and status.

Important Defense Point:

The low-stock alert is based on a clear threshold rule. It is reliable and simple to explain. It does not automatically place purchase orders.

4. Real-Time Analysis

Speech:

In my project, real-time analysis means the dashboard loads the latest available data from the backend when the page or API request is made. It is not a WebSocket-based live streaming system. The admin dashboard uses current database data for KPIs, recent orders, low-stock products, top products, audit activity, and forecast results. This gives the admin an updated operational view without manually checking database records.

How It Works:

The frontend requests dashboard analytics from the backend analytics API.

The backend calculates counts for active products, users, and orders.

It reads recent orders, low-stock products, top products, and audit logs.

It returns these values in one dashboard overview response.

Frontend dashboard pages use no-store fetching for protected dashboard data, so they request fresh data instead of relying on old cached data.

The data updates when the dashboard page is loaded or refreshed, or when the related API call is made again.

How I Built It:

The dashboard analytics logic is in backend/src/modules/analytics/analytics.service.ts.

The getDashboardOverview function prepares KPIs, revenue series, inventory alerts, top products, recent orders, and audit activity.

The analytics route is /api/analytics/overview.

This route is protected for super_admin and admin roles.

The frontend function getDashboardOverview is in frontend/lib/api.ts.

The admin dashboard page is frontend/app/(admin)/admin/page.tsx.

The frontend components include MetricCard, RevenueTrend, InventoryAlerts, OrdersPanel, ActivityFeed, and TopProducts.

Important Defense Point:

If I am asked whether it is real-time streaming, my answer is no. It is near real-time API-based analysis using fresh database reads on request. WebSocket live updates can be added in future work.

5. AI-Based Customer Support Chatbot

Speech:

The chatbot in Smart Commerce is a customer support and project help feature. It can answer questions about products, checkout, receipts, order tracking, dashboards, low-stock alerts, demand forecast, demo accounts, and setup. It is important to explain that this chatbot does not use an external AI model. It is a rule-based chatbot using a project knowledge database and keyword matching. I call it AI-based in the sense that it simulates support conversation using matching logic, but technically it is rule-based and explainable.

How It Works:

The user types a question into the chat widget.

The frontend sends the question to the /api/chatbot route.

The chatbot normalizes the text by making it lowercase, removing unnecessary characters, and applying small spelling fixes.

It removes common stop words and checks the question against known keywords.

It first tries to answer dashboard or low-stock related questions.

Then it tries to answer project feature questions from the project knowledge database.

Then it tries to answer product questions using fallback product data.

If no strong match is found, it returns a helpful default response listing the topics it can answer.

How I Built It:

The chatbot UI is in frontend/components/chatbot/chat-widget.tsx.

The API route is frontend/app/api/chatbot/route.ts.

The chatbot knowledge and matching logic are in frontend/lib/chatbot-knowledge.ts.

The knowledge database contains entries for project overview, customer flow, PDF receipt, admin dashboard, low stock, demand forecast, vendor dashboard, inventory, orders, demo accounts, tech stack, chatbot, and setup.

The chatbot uses scoring functions, keyword matching, category aliases, and formatted product responses.

The chat widget is included globally in frontend/app/layout.tsx, so it appears across the application.

Important Defense Point:

If I am asked whether the chatbot uses ChatGPT or an external LLM, my answer is no. It is a rule-based chatbot built inside the project using project knowledge and keyword scoring.

Short Combined Speech for Presentation

The key intelligent features of my Smart Commerce system are demand forecasting, inventory intelligence, low-stock alerts, real-time dashboard analysis, and the customer support chatbot.

The demand forecasting module uses previous order data and a Python script to estimate future product demand. It is explainable and lightweight. It uses weighted average, trend detection, keyword extraction, and confidence scoring.

The inventory intelligence module helps admins and vendors understand stock levels. It shows current stock, reorder points, stock status, and allows stock adjustment with inventory logs.

The low-stock alert feature identifies products below 7 units and shows product name, vendor name, SKU, current stock, suggested reorder quantity, and status.

The real-time analysis in my project is API-based. The dashboard reads the latest database data when the page or API request is made. It is not WebSocket streaming, but it gives an updated operational view.

The chatbot is a rule-based customer support assistant. It answers questions from a project knowledge database and product matching rules. It helps users ask about products, checkout, order tracking, dashboards, low stock, forecast, demo accounts, and setup.

Together, these features make the system more than a normal e-commerce website. They connect shopping, inventory, analytics, support, and forecasting into one role-based platform.

Quick Supervisor Q&A

Question: Is your forecasting deep learning?

Answer: No. It is an explainable baseline forecasting method using Python, weighted average, trend detection, keyword extraction, and confidence scoring.

Question: What data is used for forecasting?

Answer: Active products and non-cancelled orders. The backend groups order item quantities into recent weekly sales history.

Question: Who can access demand forecasting?

Answer: Admin and super-admin roles through the protected analytics route.

Question: Does the system automatically reduce stock when an order is placed?

Answer: In the current code, stock adjustment is implemented manually through inventory adjustment. Order creation does not automatically reduce product stock.

Question: Is the dashboard live with WebSocket?

Answer: No. It uses fresh API-based database reads when the dashboard or API request is loaded. WebSocket live updates can be future work.

Question: Does the chatbot use ChatGPT?

Answer: No. It is a local rule-based chatbot using project knowledge, keyword matching, and scoring.

Question: What makes inventory intelligent?

Answer: Stock visibility, reorder point comparison, low-stock alerts, suggested reorder quantity, stock adjustment, and inventory logs.